

SLR Parsing Table

09 April, 2026

Simple LR

- # The Simple LR or SLR method is the weakest method to construct an LR parsing table from a grammar. We shall refer to the parsing table constructed by the method as an *SLR table*, and to an LR parser using an SLR-parsing table as an *SLR parser*.

Items

- # An LR(0) item or item of a grammar G is a production with a $\cdot$ (dot) at some position of the body. Thus, the production $A \rightarrow XYZ$ yields the four items:

$$\begin{array}{ll} A \rightarrow \cdot XYZ & A \rightarrow X \cdot YZ \\ A \rightarrow XY \cdot Z & A \rightarrow XYZ \cdot \end{array}$$

- # Intuitively, an item indicates how much of a production we have seen at a given point in the parsing process. For example:
 - $[A \rightarrow \cdot XYZ]$ indicates that we hope to see a string derivable from XYZ next on the input.
 - $[A \rightarrow X \cdot YZ]$ indicates that we have just seen on the input a string derivable from X and that we hope to see a string derivable from YZ .
 - $[A \rightarrow XYZ \cdot]$ indicates that we have already seen a string derivable from XYZ and that maybe it is time to reduce by A .

Canonical LR(0) collection

- # One collection of sets of LR(0) items, which we call the canonical LR(0) collection, provides the basic for constructing SLR parsers.
- # To construct LR(0) collection for a grammar, we define an augmented grammar and two functions CLOSURE and GOTO.

Augmented grammar

- # If G is a grammar with start symbol S , then $\bar{G}$, the augmented grammar for G is G with a start symbol S' and the production $S' \rightarrow S$.
- # The purpose of this new starting production is to indicate to the parser when it should stop parsing and announce acceptance of the input.
- # The acceptance occurs when and only when the parser is about to reduce by $S' \rightarrow S$.

CLOSURE operation

- # If I is the set of items for a grammar G , then $\text{CLOSURE}(I)$ is the set of items constructed from I by the following two rules:
 1. Initially, every item I is added to $\text{CLOSURE}(I)$.

2. If $A \rightarrow \alpha \cdot B\beta$ is in $\text{CLOSURE}(I)$ and $B \rightarrow \gamma$ is a production, then add the item $B \rightarrow \cdot \gamma$ to $\text{CLOSURE}(I)$ if it is not already there. We apply this rule until no more new items can be added to $\text{CLOSURE}(I)$.

Intuitively, $A \rightarrow \alpha \cdot B\beta$ in $\text{CLOSURE}(I)$ indicates that, at some point in the parsing process, we think we might next see a substring derivable from $B\beta$ as input.

As an example, for a grammar with the following production rules:

$$\begin{array}{ll} E' \rightarrow E & E \rightarrow E + T \mid T \\ T \rightarrow T * F \mid F & F \rightarrow (E) \mid \text{id} \end{array}$$

If I is a set of one item $\{[E' \rightarrow \cdot E]\}$, then $\text{CLOSURE}(I)$ contains the items:

$$\begin{array}{llll} E' \rightarrow \cdot E & E \rightarrow \cdot E + T & E \rightarrow \cdot T & T \rightarrow \cdot T * F \\ T \rightarrow \cdot F & F \rightarrow \cdot (E) & F \rightarrow \cdot \text{id} & \end{array}$$

GOTO operation

$\text{GOTO}(I, X)$ is defined to be the closure of the set of all items $[A \rightarrow \alpha X \cdot \beta]$ such that $[A \rightarrow \alpha \cdot X\beta]$ is in I , where I is a set of items and X is a grammar symbol.

Intuitively, the GOTO function is used to define the transitions in the LR(0) automaton for a grammar.

Construction of SLR parsing table

Input: An augmented grammar G' .

Output: The SLR parsing table functions ACTION and GOTO for G' .

Method:

1. Construct $C = \{I_0, I_1, I_2, \dots, I_n\}$, the collection of sets of LR(0) items for G' .
2. State i is constructed from I_i . The parsing actions for state i are determined as follows:
 - a. If $[A \rightarrow \alpha \cdot a\beta]$ is in I_i and $\text{GOTO}[I_i, a] = I_j$, then set $\text{ACTION}[i, a]$ to "Shift j ". Here, a must be a terminal.
 - b. If $[A \rightarrow \alpha \cdot]$ is in I_i , then set $\text{ACTION}[i, a]$ to "Reduce $A \rightarrow \alpha$ " for all a in $\text{FOLLOW}(A)$.
 - c. If $[S' \rightarrow S \cdot]$ is in I_i , then set $\text{ACTION}[i, \$]$ to ACCEPT.

If any conflicting actions are generated by the above rules, we say that the grammar is not SLR(1), in which case the algorithm fails to produce a parser.

3. The GOTO transitions for state i is constructed for all non-terminals A using the rule:
If $\text{GOTO}[I_i, A] = I_j$, then $\text{GOTO}[i, A] = j$.
4. All entries not defined by rules 2 and 3 are made ERROR.
5. The initial state of the parser is the one constructed from the set of items containing $[S' \rightarrow \cdot S]$.

Example 1

Create SLR parsing table for a grammar with the following production rules:

$$\begin{array}{llll} E' \rightarrow \cdot E & E \rightarrow \cdot E + T & E \rightarrow \cdot T & T \rightarrow \cdot T * F \\ T \rightarrow \cdot F & F \rightarrow \cdot (E) & F \rightarrow \cdot \text{id} & \end{array}$$

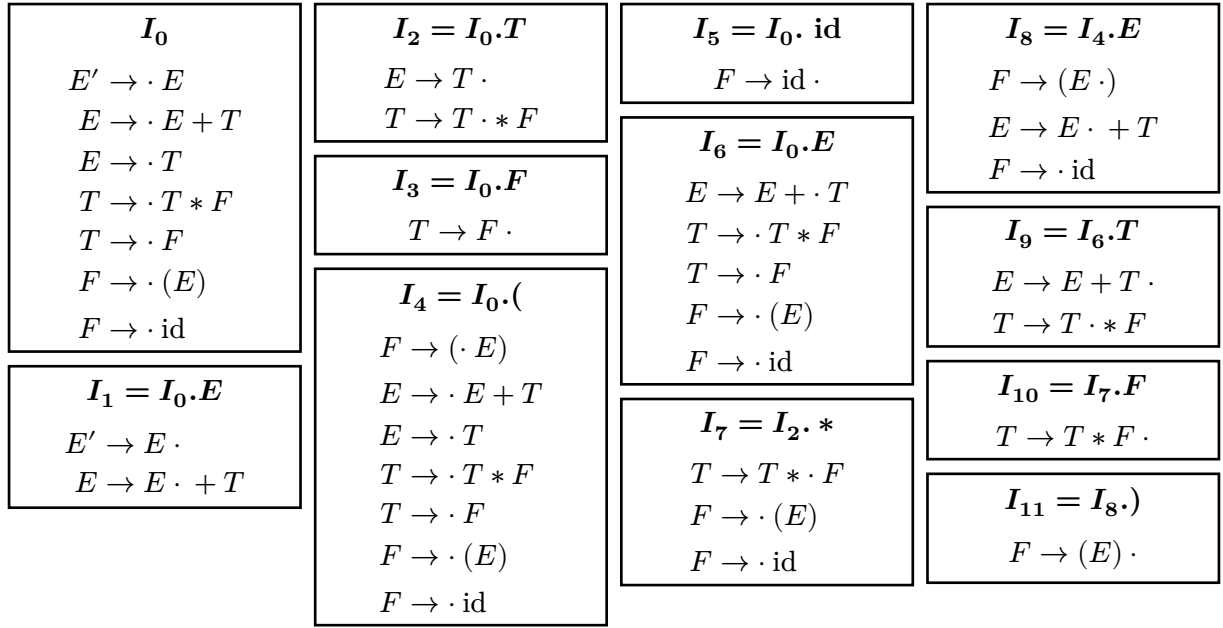


Figure 1: Canonical LR(0) collection

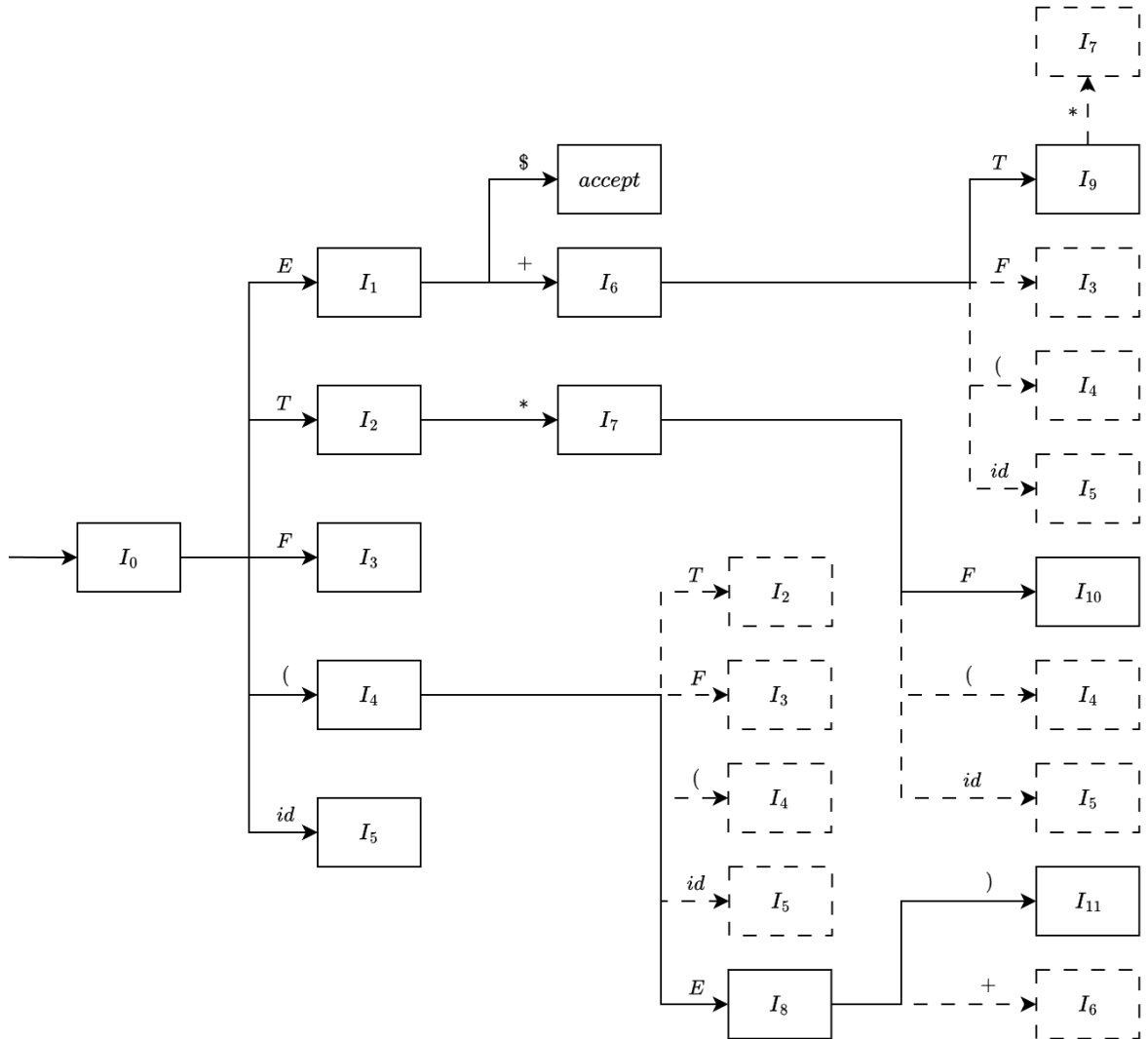


Figure 2: LR(0) automaton